

title: "Nghiên cứu repo: MemMachine/MemMachine"

repo_url: <https://github.com/MemMachine/MemMachine>

research_date: 2026-07-29

researcher: github-repo-researcher

workflow: WF-GITHUB-RESEARCH

branch: research/memmachine-2026-07-29

status: analysis-complete

RESEARCH: MemMachine/MemMachine

Lưu ý: Đây là lần nghiên cứu thứ 2 — lần đầu vào 2026-07-19 (nhánh `research/memmachine-2026-07-19`, chưa merge). Lần này cập nhật thêm các module mới phát hiện trong repo: `retrieval_agent`, `packages/skills/`, `AGENTS.md`, và CLI tool `mem-cli`.

1. Tổng quan repo

Mục đích

MemMachine là một **memory layer mã nguồn mở dành cho AI agents và ứng dụng LLM**. Mục tiêu cốt lõi là giải quyết bài toán "stateless AI" — chatbot/agent quên toàn bộ ngữ cảnh khi kết thúc session — bằng cách cung cấp lớp lưu trữ bộ nhớ bền vững, có thể tìm kiếm ngữ nghĩa.

Tagline chính thức: *"Stop building stateless agents. Give your AI persistent memory with just 5 lines of code."*

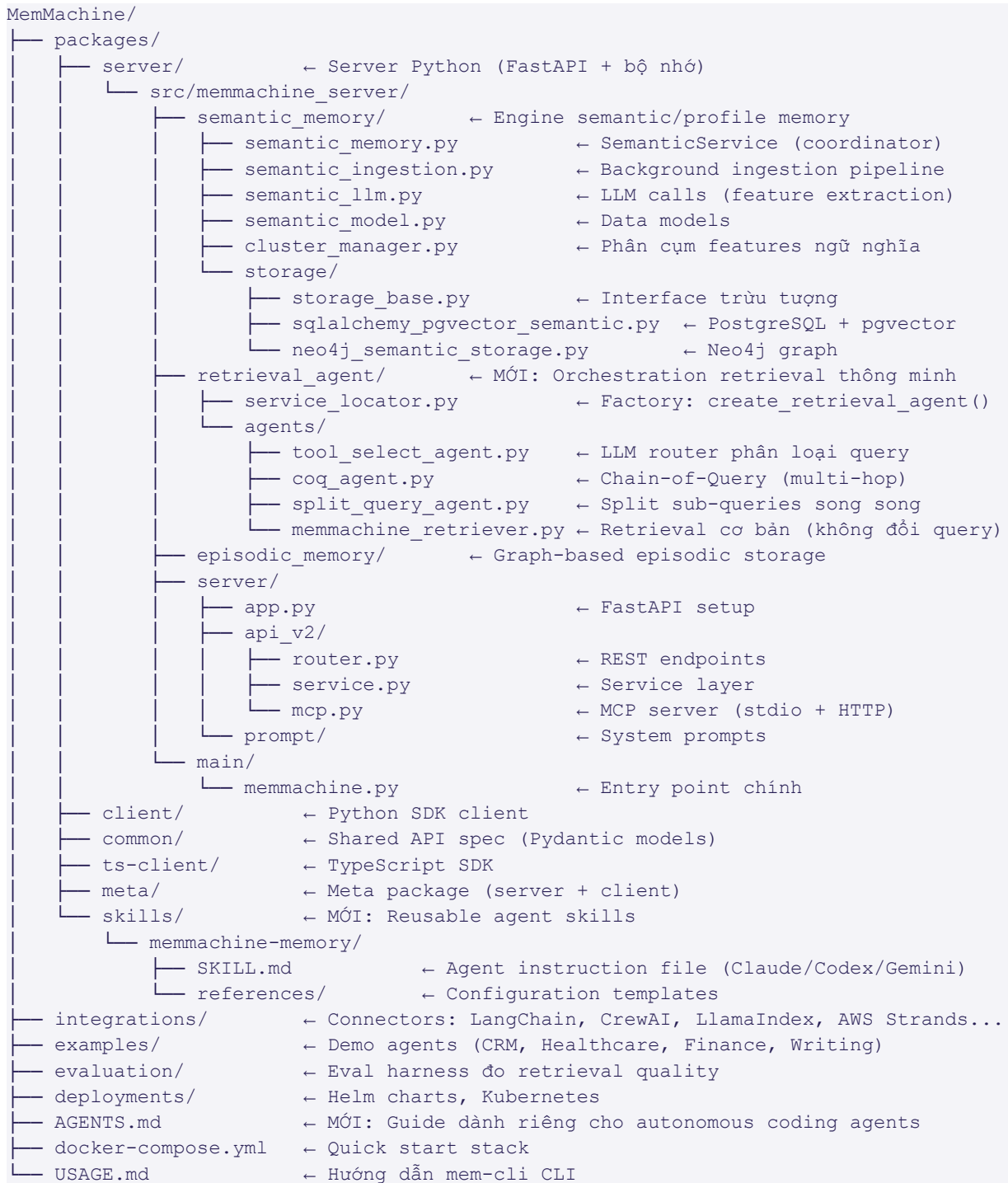
Đối tượng sử dụng

- Developers xây dựng AI agents, assistants, autonomous workflows.
- Researchers thực nghiệm với kiến trúc agent và cognitive model.
- Teams cần cross-session memory cho LLM applications.

Vấn đề giải quyết

Vấn đề	Giải pháp MemMachine
-----	-----
AI quên toàn bộ sau mỗi session	Episodic memory lưu graph các hội thoại
Không nhớ sở thích/thông tin user	Profile memory (SQL) lưu facts dài hạn
RAG chỉ phù hợp kiến thức tĩnh	Kết hợp RAG + semantic memory động
Phải gửi toàn bộ lịch sử vào context	Retrieval Agent thông minh — chỉ lấy phần liên quan
Query phức tạp multi-hop không tìm được	Chiến lược CoQ (Chain-of-Query) tự decompose

2. Cấu trúc dự án



3. Phân tích kỹ thuật

3.1 Ba loại bộ nhớ cốt lõi

MemMachine hiện thực ba tầng memory phản ánh mô hình nhận thức con người:

Working Memory (Short-Term)

- Ngữ cảnh session hiện tại — lịch sử hội thoại gần nhất.
- Không persistent, chỉ tồn tại trong session.

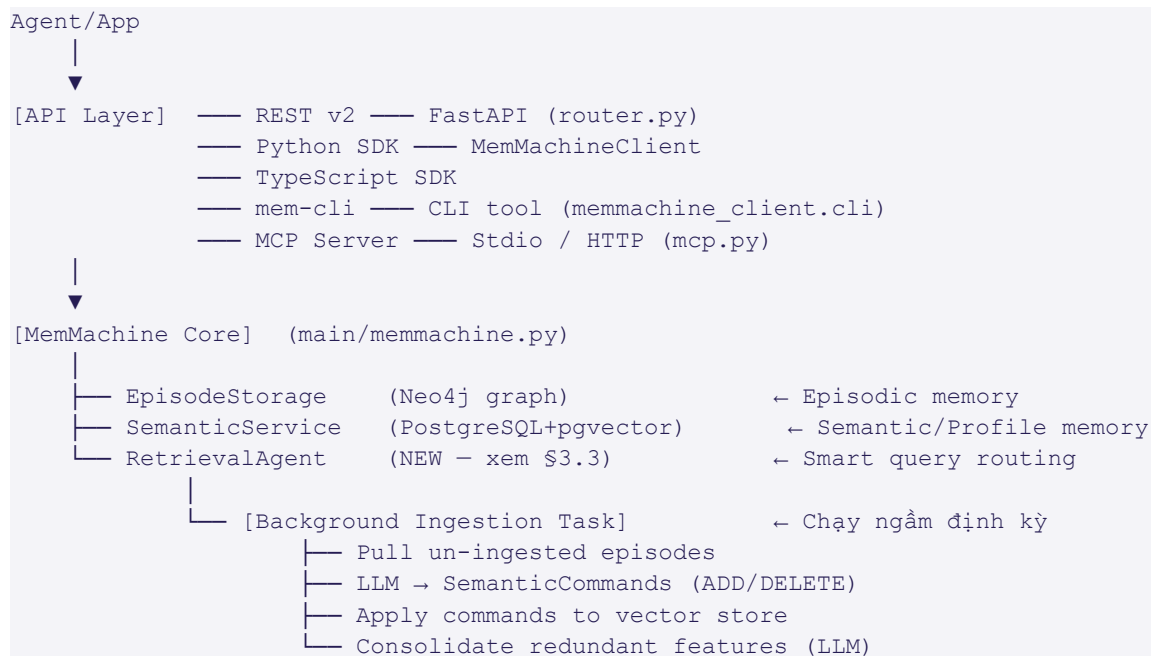
Episodic Memory (Long-Term — Graph-based)

- Lưu trữ các episode hội thoại cụ thể dưới dạng graph (Neo4j).
- Hỗ trợ cả `short_term` (recent N messages) và `long_term` (summarized).
- `EpisodeStorage` là interface trừu tượng — dễ swap backend.

Semantic/Profile Memory (Long-Term — SQL + Vector)

- `SemanticFeature` lưu facts/preferences được trích xuất từ hội thoại.
- Lưu ở PostgreSQL + pgvector (embedding similarity search).
- Phân loại theo "category" (VD: "travel preferences", "dietary restrictions") với prompt hướng dẫn LLM trích xuất.

3.2 Kiến trúc tổng thể



3.3 Retrieval Agent System — Điểm nổi bật mới nhất

Module: `packages/server/src/memmachine_server/retrieval_agent/`

Nguồn cảm hứng học thuật: Luo et al. (2025), "Agent Lightning: Train ANY AI Agents with Reinforcement Learning", arXiv:2508.03680.

Đây là điểm nổi bật lớn nhất kể từ lần nghiên cứu trước (2026-07-19). Thay vì tìm kiếm thô bằng vector similarity, MemMachine thêm một lớp **orchestration thông minh** để xử lý query phức tạp.

4 chiến lược retrieval:

Agent	Khi nào dùng	Cơ chế
---	---	---
<code>ToolSelectAgent</code>	Mặc định — router phân loại	Gửi query cho LLM, LLM chọn 1 trong 3 strategy dưới đây
<code>ChainOfQueryAgent</code> (CoQ)	Query multi-hop (phụ thuộc tuần	Lặp: search → kiểm tra đủ? → viết

	tự)	query tiếp nếu chưa đủ
SplitQueryAgent	Query 1 bước nhưng nhiều entity	Split thành N sub-queries → chạy song song → merge
MemMachineAgent	Query đơn giản, trực tiếp	Tìm kiếm không đổi query

Flow của ToolSelectAgent (tool_select_agent.py):

Query → LLM phân loại → 1 trong 3 loại:

- A) MULTI-HOP (chuỗi phụ thuộc) → ChainOfQueryAgent
- B) SINGLE-HOP + MULTI-ENTITY (độc lập) → SplitQueryAgent
- C) SINGLE-HOP / DIRECT (đơn giản) → MemMachineAgent

Ví dụ phân loại (từ prompt thật trong code):

- "Who is the author of 'Dune'?" → MemMachineAgent (trực tiếp)
- "Give the capitals of Spain and Portugal." → SplitQueryAgent (2 entity độc lập)
- "Find the spouse of Marie Curie, then name his primary field." → ChainOfQueryAgent (cần kết quả bước 1 để làm bước 2)

Chain-of-Query (CoQ) sufficiency check (coq_agent.py):

```
# Vòng lặp:
# 1. Search với current_query
# 2. LLM kiểm tra: retrieved_episodes đủ để trả lời original_query chưa?
#    → is_sufficient + evidence_indices + confidence_score + new_query
# 3. Nếu chưa đủ → dùng new_query, lặp lại
# 4. Dừng khi đủ hoặc max_iterations
```

Đây là pattern **retrieval-augmented generation với vòng lặp tự đánh giá** — khác với RAG truyền thống chỉ search 1 lần.

Kích hoạt `--agent-mode`: User có thể opt-in vào retrieval thông minh qua CLI:

```
mem-cli memory search "project decision about retrieval limits" --agent-mode
```

3.4 Skills Package — Agent Skill cho AI Coding Agents

File: packages/skills/memmachine-memory/SKILL.md

Đây là pattern hoàn toàn mới: MemMachine cung cấp một **"skill"** (file instruction markdown) có thể được cài vào AI coding agents (Claude Code, Codex, Gemini, OpenCode) để dạy agent cách dùng MemMachine làm memory backend.

Nguyên tắc cốt lõi của skill:

- Khi agent cần tìm thông tin về lịch sử/quyết định/preferences → **PHẢI** gọi `mem-cli` **TRƯỚC** khi dùng `grep`, `find`, hay `ls`
- Repository search trả lời "file nào đang tồn tại"; MemMachine trả lời "quyết định/preference nào đã được ghi nhớ"
- Simple Query Rule: mỗi query phải single-hop, trực tiếp trả lời được — nếu phức tạp thì decompose

Cài đặt:

```
npm skills add https://github.com/MemMachine/MemMachine \
  --skill packages/skills/memmachine-memory
# hoặc
pipx run shskills install --url https://github.com/MemMachine/MemMachine \
  --agent claude --subpath packages/skills/memmachine-memory
```

Retrieval Workflow của skill (7 bước):

1. Quyết định có cần retrieval không (context hiện tại có đủ không?)
2. TRƯỚC khi grep/find → chạy `mem-cli memory search`
3. Viết một query đơn giản, single-hop
4. Chạy query với limit nhỏ
5. Kiểm tra JSON trả về
6. Đủ → dừng; Chưa đủ → query tiếp cho phần còn thiếu
7. Tối đa 3 queries không đủ → đặt giả định hoặc hỏi user

3.5 AGENTS.md — Guide dành cho Autonomous Coding Agents

MemMachine có file `AGENTS.md` với hướng dẫn cho coding agents tự chạy task trong repo: build commands (`uv sync`, `ruff check`, `ty check`), test commands (`pytest`, `-k keyword`, `-m mark`), và style conventions. Đây là pattern ngày càng phổ biến trong open-source repos để tối ưu hoạt động của AI coding agents.

3.6 Background Ingestion Pipeline

File: `packages/server/src/memmachine_server/semantic_memory/semantic_ingestion.py`

Pattern quan trọng: **background ingestion không blocking:**

1. Agent gửi episode qua `add_episodes()` — trả về ngay, không chờ xử lý.
2. Background task chạy định kỳ (mặc định poll mỗi 2 giây) — tìm `set_id` có `uninged_messages >= 5` HOẶC `age >= 5` phút.
3. Với mỗi `set_id` "dirty": pull episodes → LLM sinh `SemanticCommands` (ADD/DELETE) → apply vào vector store → purge rows đã ingest.
4. Nếu `feature_count > threshold (20)` → consolidation (LLM gộp/xoá duplicate).
5. Backoff khi lỗi: `backoff_sec = min(backoff_sec * 2, 60.0)` — tránh hammering.

3.7 Parallel Vector Search

File: `packages/server/src/memmachine_server/semantic_memory/semantic_memory.py`

Khi search nhiều `set_id` cùng lúc, hệ thống fan-out song song và merge kết quả streaming:

```
iterators = [self._set_id_search(set_id=sid, embedding=emb, ...)
              for sid, emb in zip(set_ids, embeddings)]
async for feature in merge_async_iterators(iterators):
    yield feature
```

Không chờ từng cái xong mới xử lý tiếp — kết quả streaming ngay khi sẵn sàng.

3.8 MCP Native Server

File `packages/server/src/memmachine_server/server/api_v2/mcp.py` cung cấp MCP server dùng `fastmcp`:

- `memmachine-mcp-stdio` — cho Claude Desktop, Cursor (stdin/stdout).
- `memmachine-mcp-http` — cho web clients.
- MCP tools expose: `add_memory`, `search_memory`, `delete_memory` — agent LLM gọi trực tiếp trong conversation.

3.9 Điểm mạnh / Điểm yếu

	Điểm mạnh	Điểm yếu
---	---	---

Retrieval	Retrieval Agent thông minh: CoQ/Split/ToolSelect	Thêm LLM calls → tăng latency và chi phí
Architecture	LLM-agnostic, self-hosted được, MCP native	Infra phức tạp: Neo4j + PostgreSQL + pgvector
DX	Python SDK gọn (5 dòng), CLI <code>mem-cli</code> , Agent Skill	Không có TTL tự động cho memory cũ
Ingestion	Background, non-blocking, exponential backoff	Latency 2 giây poll — không realtime
Academic	Có backing nghiên cứu (Luo et al. 2025 arXiv)	Nghiên cứu còn mới (2025), chưa battle-tested ở scale

4. Công nghệ / Stack

Thành phần	Công nghệ
---	---
Backend server	Python 3.12+, FastAPI, asyncio
Package manager	uv (UV workspace monorepo)
Episodic memory	Neo4j (graph database)
Semantic memory	PostgreSQL + pgvector
DB migrations	Alembic
Embeddings	Configurable (OpenAI, Bedrock, Ollama, local)
LLM	Configurable (OpenAI, Anthropic, Bedrock, Ollama)
SDK	Python (<code>memmachine-client</code>) + TypeScript
CLI	<code>mem-cli</code> (alias cho <code>memmachine_client.cli</code>)
MCP server	fastmcp
Agent Skill format	SKILL.md (markdown instruction) — <code>npm skills add</code>
Containerization	Docker, Docker Compose, Kubernetes (Helm)
Lint/Format	Ruff
Type check	ty
Test	pytest + pytest-asyncio, testcontainers
Multi-framework	LangChain, LangGraph, CrewAI, LlamaIndex, AWS Strands, n8n, Dify, FastGPT

5. Thông tin repo

Thuộc tính	Giá trị
---	---
License	Apache 2.0
Ngôn ngữ chính	Python (server/SDK/CLI), TypeScript (ts-client)
Python version	3.12+
Hoạt động	Đang phát triển tích cực — nhiều module mới từ tháng 7/2026 (retrieval_agent, skills)
Academic backing	Luo et al. (2025), arXiv:2508.03680 "Agent Lightning"
MCP support	Native (stdio + HTTP)
Cloud option	Có — <code>console.memmachine.ai</code>
Community	Discord https://discord.gg/usydanvkqD , GitHub Discussions

6. Hiện trạng KZTEK

Cơ chế "memory" hiện tại của KZTEK

KZTEK workspace là hệ thống AI Agent Creator — các agent (Claude Code) chạy trong session ngắn, không có persistence cross-session ngoại trừ:

Cơ chế	File	Mục đích
---	---	---
Handoff Log	<code>docs/plans/PLAN-*/steps/STEP-*.md</code> (mục Handoff Log)	Agent bước N ghi tóm tắt → bước N+1 đọc để không nghiên cứu lại
Progress Ledger	<code>_workspace/progress.md</code> (git-ignored)	Append-only log nhẹ — phục hồi nhanh khi compact
GOTCHAS.md	<code>.claude/shared/GOTCHAS.md</code>	Lỗi ngầm đã biết — "long-term memory" dạng human-curated
Plan File	<code>docs/plans/PLAN-*/PLAN-MASTER.md</code>	Tiến độ task, trạng thái bước, artifacts
LESSONS.md	<code>docs/LESSONS.md</code>	Bài học workflow và business decision (mới thêm 2026-07-29)
lessons/ toàn cục	<code>C:\Users\nguye\.claude\lessons\</code>	Lesson kỹ thuật per-category, dùng chung mọi project

Không có:

- Lưu trữ hội thoại cross-session (mỗi session bắt đầu từ đầu).
- Profile agent (agent không nhớ style/preference của user từ session trước).
- Vector search trên lịch sử tương tác.
- Graph relationship giữa concepts đã thảo luận.
- MCP server riêng cho memory.
- CLI tool để query memory.

Tóm lại: KZTEK dùng file markdown + git làm "bộ nhớ dài hạn" — đủ cho workflow tài liệu/code, nhưng không có semantic retrieval hay user profiling.

Đọc tham chiếu thực tế:

- `.claude/shared/GOTCHAS.md` — 6 entries (G001–G006), phân loại phẳng, không có category tag
- `docs/plans/` — cấu trúc MASTER + step files
- `.claude/lessons/INDEX.md` — lesson per-category kỹ thuật (toàn cục)

7. So sánh: MemMachine vs Hiện trạng KZTEK

Điểm so sánh	MemMachine	KZTEK hiện tại
---	---	---
Persistence cross-session	Graph DB (Neo4j) + SQL (PostgreSQL)	Plan file .md trong git, đọc thủ công qua Handoff Log

User profiling	Profile memory — facts/preferences per user	Không có
Semantic search	Vector similarity (pgvector)	Không có — chỉ Grep/Read text thuần
Query thông minh	Retrieval Agent: CoQ / Split / ToolSelect	Không có — agent tự Glob/Grep theo task
Multi-hop query	ChainOfQueryAgent: sufficiency loop	Không có — agent phải tự đọc nhiều file
Ingestion	Background pipeline, LLM-driven	Thủ công — Handoff Log viết bởi agent
Recovery compact	Không áp dụng	<code>_workspace/progress.md</code> (file nhẹ, append-only)
Loại thông tin lưu	Episodes (hội thoại) + semantic features	Bước task, artifact, lỗi đã biết
MCP integration	Native MCP server	Không có
Agent Skill	<code>packages/skills/memmachine-memory/SKILL.md</code> — cài 1 lệnh	Chưa có skill dạng này
AGENTS.md	Có — hướng dẫn agent tự build/test	Có — CLAUDE.md (đầy đủ hơn, dành riêng cho hệ thống agent KZTEK)
CLI tool	<code>mem-cli</code> — query/add/delete memory	Không có
Infrastructure	Neo4j + PostgreSQL + LLM — phức tạp	Chỉ git + filesystem — đơn giản
Chi phí	LLM call mỗi ingestion cycle	Không tốn token cho "memory"

8. Bảng đề xuất cải tiến (Mode A)

Bốn đề xuất dưới đây rút ra từ pattern kỹ thuật cụ thể trong MemMachine, đối chiếu trực tiếp với hiện trạng KZTEK đã khảo sát ở §6-7. KHÔNG tự áp dụng — chờ user xác nhận ở Bước 4.

#	Đề xuất	Hiện trạng KZTEK	Học từ đâu (file/pattern MemMachine)	Lý do thay đổi	Áp dụng vào đâu trong KZTEK	Đạt được gì	Rủi ro/ Effort
-	---	---	---	---	---	---	---
-							
-							
E 1	Category tagging cho GOTCHAS.md — thêm category label vào từng entry và bảng lọc nhanh ở đầu file	GOTCHAS.md có 6 entries (G001–G006) với mục lục phẳng theo số thứ tự — không có tag phân loại. Agent muốn tìm lỗi loại "script" phải đọc tuần tự tất cả 6 entries	<code>packages/server/src/memmachine_server/semantic_memory/config_store/</code> — category system: mỗi <code>set_id</code> có danh sách category (" <code>travel preferences</code> ", " <code>dietary restrictions</code> ") với prompt hướng dẫn LLM trích xuất đúng loại. Pattern: gán category → tìm theo category thay vì scan full	GOTCHAS.md với 6 entries hiện chưa gây vấn	<code>.claude/sharded/GOTCHAS.md</code> — (a) thêm bảng lọc nhanh ở đầu file với cột <code>Category: [SCRIPT] [CONFIG] [ENCODING] [GIT] [AGENT-LOOP]</code> ; (b)	Khi gặp <code>ModuleNotFoundError</code> → tra GOTCHAS.md filter <code>[SCRIPT]</code> → đọc 1–2 entries thay vì 6.	Rủi ro: rất thấp — chỉ thêm metadata text

				đề. Như ng khi đạt 15+ entr ies, Disp atch er phải đọc 100 % nội dun g để tìm entr y liên qua n đến loại lỗi cụ thể — khô ng có cách lọc theo dom ain (scri pt, git, conf ig, enc odin g...)	thêm dòng Category: [xxx] vào header mỗi entry hiện có và template entry mới ở comment cuối file. Cập nhật .claude /shared/CO RE.md §6 để nhắc tra theo category trước	Khi có 15+ entries tổng, số entries phải đọc per lookup giảm từ ~100% xuống ~15- 25% (chỉ đọc đúng category)	, khô ng đổi logi c nào . Effo rt: rất thấ p — sửa 6 ent ry hea der s + thê m bản g lọc + cập nhậ t tem plat e co mm ent; 1 ses sio n < 20 phú t
E 2	Sufficienc y tracking mở rộng cho §17.1 CODE- GRAPH — sau khi đọc CODE- GRAPH, ghi rõ câu	CLAUDE.md §17.1 có rule "≥ 3/5 câu trả lời được từ CODE-GRAPH → bắt đầu coding". Nhưng thiếu bước "tracking câu nào còn	packages/server/src/memmachine_server/retrieval_agent/agents/coq_agent.py — CoQ sufficiency check trả về: is_sufficient (bool) + evidence_indices (phần đã đủ) + new_query (câu hỏi tiếp theo cho phần THIẾU). Pattern: không chỉ hỏi "đủ chưa?" mà còn track "đang thiếu gì cụ thể?" để	Khô ng có trac king rõ ràng . Sau khi COD	CLAUDE.md §17.1 bước 4 — thêm sub- step sau "Thiếu thông tin → chỉ đọc thêm file/module cụ thể": "Ghi rõ câu nào trong	Sau khi CODE- GRAPH trả lời 4/5 câu, agent đọc thêm đúng 1 file thay	Rủi ro: rất thấ p — chỉ thê m hư

	nào trong 5 câu còn thiếu, chỉ đọc source file của câu đó	thiếu → chỉ đọc source file liên quan đến câu đó". Agent hay đọc thêm nhiều source files "để chắc" thay vì nhầm đúng phần còn thiếu	query chính xác bước tiếp theo	E- GRA PH trả lời đượ c 4/5 câu, age nt đọc thê m sour ce files khô ng theo ngu yên tắc nào — có thể đọc 3-5 files để chắc chắ n, dù chỉ cần đọc 1 file cho câu (d) còn thiế u. Tok en lãng phí và cont ext win dow phìn	5 câu chưa được trả lời → chỉ đọc ĐÚNG source file liên quan đến câu đó (không đọc file khác). VD: câu (d) 'API endpoint ở file:line nào?' chưa trả lời → chỉ đọc file router/api, không đọc service/model/test."	vì 3-5 files. Trên task đọc codebase lần đầu (CODE-GRAPH mới), tiết kiệm 2-4 Read tool calls per task — tương đương giảm ~200-800 token context window per coding session	ống dẫn rõ hơn vào §17.1, không đổi rule bắt buộc nào. Effort: rất thấp — thêm 3-5 dòng vào §17.1 CLAUD E.md
--	---	---	--------------------------------	---	--	---	--

				h khô ng cần thiế t			
E 3	Simple Query Rule cho lessons lookup — thêm rule "bắt đầu với 1 file, dừng khi đủ, tối đa 3 lookups" vào quy trình đọc lessons	C:\Users\n guye\.clau de\CLAUDE. md \$Khi nào đọc lessons yêu cầu: Glob → đọc INDEX.md → đọc category liên quan. Không có rule "dừng khi đã tìm được thông tin cần" hay "phân loại nhu cầu trước khi đọc nhiều file". Agent thường đọc toàn bộ category folder (VD: tất cả 5 files trong avalonia/) kể cả khi chỉ cần 1 lesson	packages/skills/memmach ne-memory/SKILL.md — Simple Query Rule: "Retrieve with simple queries, check whether each result is sufficient, and only continue querying for the next missing fact." + "Stop when sufficient. If three simple queries do not retrieve sufficient context, proceed with an explicit assumption or ask the user." Pattern: query one-fact-at-a-time + sufficiency gate sau mỗi query	Tas k aval onia hiện tại: age nt đọc IND EX. md → thấy cate gory ava lon ia/ → đọc TẤT CẢ 5 files dù chỉ cần trán h lỗi reso urce path cụ thể. Khô ng có "suff icie ncy gate " giữa các file đọc → đọc thứ a 3-	C:\Users\n guye\.clau de\CLAUDE. md \$Khi nào đọc lessons — bổ sung sub- rule sau dòng "đọc lesson liên quan": (a) Đọc 1 file rõ nhất liên quan trước (không đọc cả folder). (b) Đủ để tránh lỗi đã biết cho task này? → Dừng. (c) Còn thiếu gì cụ thể? → Đọc thêm đúng 1 file có chứa thông tin đó. (d) Tối đa 3 lookups — nếu vẫn thiếu thì bắt đầu task, ghi lesson mới sau nếu gặp lỗi.	Task avalonia → agent đọc avalonia/ava lonia-resour ce- path.md (1 file) thay vì toàn bộ avalonia/ folder (5 files). Per task, context Pre-0 giảm 3- 4 Read tool calls → tiết kiệm ~400- 800 token. Với 20 tasks/ng ày, tích lũy tiết kiệm đáng kể. Thêm: không bị "lesson noise" từ files không liên quan	Rủi ro: thấ p — nếu rule quá chặ t, age nt có thể bỏ sót less on qua n trợ ng. Miti gat e: dùn g tối đa 3 loo kup s (kh ông phả i 1) + luô n đọc IND EX. md trư ớc để biết cate gor

				4 less ons không liên quan đến task. Với 50+ less ons toàn cục, overhead tăng tuyến tính theo thời gian			y. Effort: rất thấp — thêm 4–5 dòng vào CLAUDE.md toàn cục
E 4	Compact pre-coding SKILL file theo pattern SKILL.md — tạo <code>.claude/commands/pre-coding-check.md</code> là skill ngắn gọn cho subagents trong session isolation	Khi Dispatcher tạo prompt cho subagent (§16.5 Bước 2a), phải manually nhúng context từ CLAUDE.md vào prompt — dài và error-prone. Không có chuẩn "compact skill" tự chứa đủ hướng dẫn pre-coding checks. Subagents hay bỏ sót bước đọc CODE-GRAPH, lessons, GOTCHAS vì không được nhắc trong prompt ngắn	<code>packages/skills/memmachine-memory/SKILL.md</code> — skill ~200 dòng, self-contained, định nghĩa: setup check → retrieval workflow (7 bước) → simple query rule → adding memory → evidence discipline. Cài bằng 1 lệnh. Pattern: compact instruction file, subagent invoke skill này thay vì đọc toàn bộ docs	Subagent nhận prompt từ Dispatcher thư ờng thiếu hướng dẫn về: đọc CODE-GRAPH trước source files, dùn	Tạo <code>.claude/commands/pre-coding-check.md</code> — skill ~60–80 dòng, self-contained gồm: (1) Check CODE-GRAPH → trả lời ≥3/5 câu → (2) Check lessons: INDEX.md → 1 file phù hợp → dừng nếu đủ → (3) Check GOTCHAS: filter category → đọc entry liên quan → (4) Start coding. Dispatcher include reference đến skill này trong mọi subagent prompt (1 dòng)	Prompt của mỗi subagent giảm ~150–300 token (bỏ phần nhắc quy trình lặp lại); pre-coding checks nhất quán 100% giữa các bước và session; khi thay đổi quy trình chỉ sửa 1 file skill, không sửa từng prompt. Subagents ít bỏ sót steps	Rủi ro: thấp-trung — nếu skill thiết u context cụ thể của task, subagent có thể hiểu sai. Mitigate:

				g đọc less ons khi đủ, tra GOT CHA S theo cate gory . Disp atch er phải viết lại hướ ng dẫn này mỗi lần tạo sub age nt pro mpt — khô ng nhất quá n giữ các bướ c và các sess ion		quan trọng	skil l chỉ chứ a ngu yên tắc chu ng, con text tas k vẫn nhú ng riên g vào pro mpt . Effo rt: thấ p — viết 1 file mới ~7 0 dòn g; cập nhậ t §16 .5 CLA UD E.m d để nhắ c Dis pat che r refe ren ce skil l
--	--	--	--	--	--	---------------	---

9. Trạng thái áp dụng đề xuất

(Điền sau khi user xác nhận ở Bước 4)

#	Đề xuất	Trạng thái
---	-----	-----
E1	Category tagging cho GOTCHAS.md	✓ Đã áp dụng — .claude/shared/GOTCHAS.md (bảng lọc + 6 category labels) + .claude/shared/CORE.md §6
E2	Sufficiency tracking mở rộng cho §17.1 CODE-GRAPH	✓ Đã áp dụng — CLAUDE.md §17.1 bước 4 (sufficiency tracking sub-step)
E3	Simple Query Rule cho lessons lookup	✓ Đã áp dụng — C:\Users\nguye\.claude\CLAUDE.md §Khi nào đọc lessons (Simple Query Rule + bảng lọc)
E4	Compact pre-coding SKILL file	✓ Đã áp dụng — tạo .claude/commands/pre-coding-check.md (mới) + CLAUDE.md §16.5 Bước 2a (1 dòng reference)